

TECHNICAL REPORT

Aluna: Suiany Pinto Gomes – 582147

Aluna: Dayane Magalhães Ferreira – 587972

1. Introdução

Neste projeto foi utilizado o dataset Dry Bean Dataset, disponível no UCI Machine Learning Repository. Esse conjunto de dados contém informações extraídas de imagens de diferentes tipos de grãos de feijão secos, sendo amplamente utilizado em pesquisas relacionadas à classificação automática de padrões agrícolas.

O dataset possui 13.611 amostras distribuídas em 7 classes distintas de feijões:

- BARBUNYA
- BOMBAY
- CALI
- DERMASON
- HOROZ
- SEKER
- SIRA

Cada amostra é composta por 16 atributos numéricos relacionados às características geométricas dos grãos, extraídos por técnicas de processamento digital de imagens, como área, perímetro, comprimento do eixo principal, comprimento do eixo secundário, circularidade, compactação e fatores de forma. Essas variáveis foram obtidas por meio de técnicas de processamento de imagens aplicadas sobre fotografias dos grãos.

Durante o desenvolvimento do projeto foram realizadas etapas de exploração e tratamento do dataset, análise de correlação entre atributos, divisão entre conjuntos de treino e teste, implementação do modelo KNN, estudo da influência do parâmetro K, aplicação de validação cruzada estratificada e utilização de Grid Search para seleção automática dos melhores hiperparâmetros.

Por fim, os resultados obtidos foram analisados criticamente com o objetivo de verificar a eficiência do modelo na classificação automática dos grãos e compreender os fatores que impactam diretamente no desempenho do algoritmo.

2. Observações

A análise exploratória inicial revelou que o conjunto de dados é perfeitamente consistente em sua estrutura de preenchimento, contendo 13.611 registros válidos com

nenhuma ocorrência de valores nulos em qualquer uma das 17 colunas de dados brutos. No entanto, duas características estruturais importantes foram identificadas durante o pré-processamento dos dados e exigiram tratamento técnico específico.

A primeira refere-se à variável-alvo *Class*. Originalmente, essa coluna era representada por dados textuais do tipo objeto, contendo strings correspondentes aos nomes das espécies de feijão, como “SEKER” e “SIRA”. Para possibilitar a utilização do dataset nos algoritmos de Aprendizado de Máquina, a variável foi convertida em rótulos numéricos inteiros por meio da técnica de codificação de etiquetas (*LabelEncoder*). Dessa forma, cada classe passou a ser representada por um valor numérico específico, garantindo compatibilidade com os modelos matemáticos implementados pelas bibliotecas do Python.

A segunda característica observada está relacionada à sensibilidade do algoritmo KNN em relação à escala dos atributos. Como o modelo realiza classificações utilizando métricas de distância, atributos com magnitudes maiores poderiam exercer influência desproporcional nos cálculos. Para evitar esse problema, foram aplicadas diferentes estratégias de normalização, incluindo padronização Z-score (*StandardScaler*), escalonamento Min-Max e transformação logarítmica combinada à padronização Z-score, permitindo uma comparação mais equilibrada entre as variáveis do dataset.

3. Resultados e discussão

3.1 Carregamento e Exploração Inicial do Dataset

A primeira etapa consistiu na leitura do arquivo *Dry_Bean_Dataset.xlsx* com a biblioteca *pandas*. Em seguida, foram utilizados os métodos *head()*, *info()* e *describe()* para inspecionar a estrutura dos dados. A função *head()* permitiu visualizar as cinco primeiras linhas do dataset, evidenciando que todos os atributos de entrada são numéricos. Já *info()* confirmou a existência de 13.611 registros, com 16 atributos numéricos (14 do tipo *float64* e 2 do tipo *int64*), além de uma coluna categórica (*Class*) do tipo *object*. Por fim, *describe()* apresentou estatísticas descritivas, como média, desvio padrão, mínimo e máximo de cada variável. Essa análise inicial foi fundamental para compreender a natureza do problema e verificar que se trata de um problema de classificação multiclasse.

3.2 Identificação e Tratamento de Valores Ausentes

Após a exploração inicial, foi realizada a verificação de valores ausentes utilizando o método *isnull().sum()*, que contabiliza a quantidade de valores nulos em cada coluna do dataset.

A análise mostrou que todas as colunas apresentaram valor zero para a contagem de elementos ausentes. Portanto, conclui-se que o conjunto de dados está completo e não contém registros faltantes. Essa característica é bastante positiva, pois elimina a necessidade de técnicas adicionais de tratamento, como imputação por média, mediana ou remoção de amostras.

Embora o código incluía o comando `dropna()`, essa operação não removeu nenhuma linha, uma vez que o dataset já estava íntegro. Assim, o número total de observações permaneceu em 13.611 amostras, distribuídas em 17 colunas.

Em seguida, a coluna `Class`, originalmente composta por nomes textuais das variedades de feijão, foi convertida para valores inteiros por meio da classe `LabelEncoder` da biblioteca `scikit-learn`. Esse procedimento é necessário porque algoritmos de aprendizado de máquina trabalham internamente com representações numéricas.

Por exemplo, as classes podem ser mapeadas da seguinte forma:

- BARBUNYA → 0
- BOMBAY → 1
- CALI → 2
- DERMASON → 3
- HOROZ → 4
- SEKER → 5
- SIRA → 6

A codificação não altera o significado das classes, apenas cria uma representação numérica equivalente.

O fato de o dataset não apresentar valores ausentes é uma vantagem importante, pois reduz a complexidade do pré-processamento e garante que todas as amostras estejam disponíveis para o treinamento do modelo. Em problemas reais, é comum encontrar dados incompletos, o que pode introduzir ruído e comprometer o desempenho do classificador.

Além disso, a transformação da variável-alvo em números inteiros tornou o conjunto de dados compatível com o algoritmo KNN, preservando a relação entre cada amostra e sua classe correspondente.

3.3 Análise de Correlação Linear e Identificação de Relevância

Nesta etapa, o objetivo foi mapear as interdependências entre as características geométricas dos grãos de feijão e identificar quais atributos apresentam maior associação estatística com a variável-alvo. Para isso, calculou-se a matriz de correlação

linear de Pearson entre todas as variáveis numéricas do conjunto de dados, seguida pelo isolamento e extração dos valores absolutos que medem a associação de cada recurso morfológico com a coluna de classificação.

Os resultados mostraram que os atributos com maior relação com a variável-alvo foram:

- Perimeter;
- EquivDiameter;
- ConvexArea;
- Area;
- MinorAxisLength.

Essa abordagem analítica permitiu estabelecer um ranking fidedigno dos atributos mais discriminantes para o processo de modelagem.

Visualização por Mapa de Calor na Figura 1.

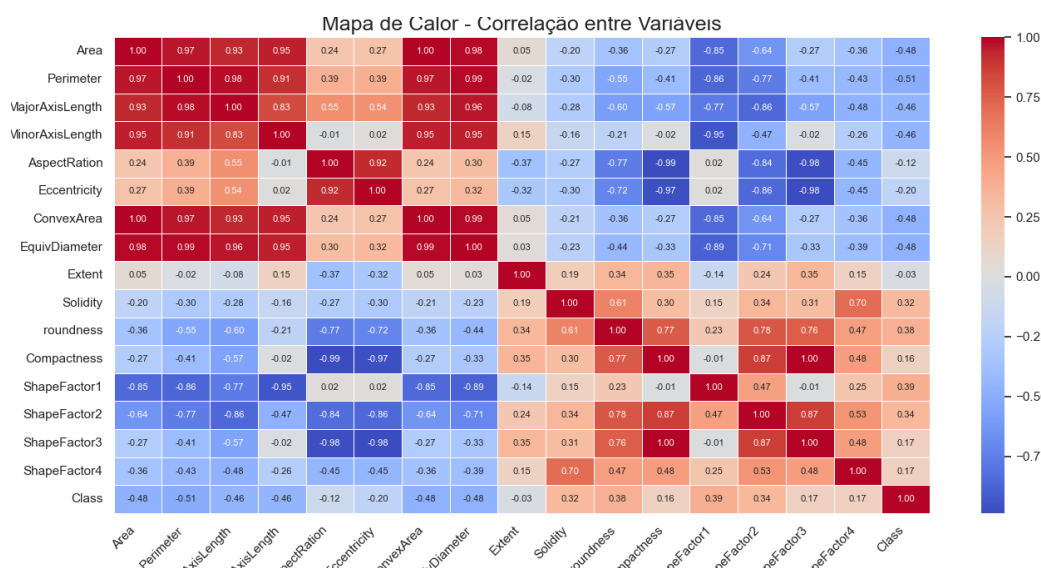


Figura 1 – Mapa de calor da correlação entre os atributos do dataset Dry Bean

O gráfico revelou que as variáveis relacionadas às dimensões físicas dos grãos apresentam uma forte colinearidade interna, destacando-se o bloco formado por: área, perímetro, maior comprimento de eixo, menor comprimento de eixo, convexidade da área e diâmetro equivalente. O comportamento fortemente correlacionado deste grupo é biologicamente natural, uma vez que o crescimento macroscópico da semente expande simultaneamente a sua área espacial, o seu perímetro e as extensões dos seus eixos lineares.

3.4 Separação entre Variáveis de Entrada e Variável-Alvo

Após a etapa de análise exploratória e identificação dos atributos relevantes, o conjunto de dados foi dividido entre variáveis independentes (*features*) e variável dependente (*target*).

As variáveis de entrada foram compostas pelos 16 atributos geométricos e morfológicos dos grãos, incluindo medidas como área, perímetro, comprimento dos eixos principal e secundário, circularidade, compactação e fatores de forma. Já a variável-alvo foi definida como a coluna *Class*, responsável por indicar a categoria correspondente de cada amostra de feijão.

Em seguida, realizou-se a separação dos dados em conjuntos de treinamento e teste utilizando a função `train_test_split()` da biblioteca Scikit-Learn, adotando a proporção de 80% para treinamento e 20% para teste, conforme prática amplamente utilizada em problemas de classificação supervisionada.

Além disso, foi definido o parâmetro `random_state = 42`, garantindo reprodutibilidade dos experimentos, isto é, permitindo que a mesma divisão fosse obtida em execuções futuras do código. Também foi utilizado o parâmetro `stratify = y`, com o objetivo de preservar proporcionalmente a distribuição original das classes em ambos os subconjuntos, evitando possíveis distorções na representatividade das categorias.

Após a divisão, o conjunto de treinamento passou a conter 10.888 amostras, enquanto o conjunto de teste foi composto por 2.723 observações.

Essa etapa foi fundamental para assegurar uma avaliação confiável do desempenho do modelo, uma vez que o algoritmo foi treinado em uma parcela dos dados e posteriormente avaliado em observações inéditas, reduzindo o risco de sobreajuste (*overfitting*) e permitindo medir sua capacidade de generalização.

3.5 Aplicação das Técnicas de Normalização

Como o algoritmo K-Nearest Neighbors (KNN) realiza classificações com base em medidas de distância entre amostras, a escala dos atributos exerce forte influência sobre o desempenho do modelo. Variáveis com magnitudes maiores podem dominar os cálculos de distância, reduzindo a contribuição de atributos com escalas menores e comprometendo a qualidade da classificação.

Com o objetivo de minimizar esse problema, foram aplicadas diferentes estratégias de normalização dos dados, permitindo avaliar o impacto de cada técnica no desempenho do algoritmo.

A primeira técnica utilizada foi a padronização Z-score, implementada por meio da classe `StandardScaler`. Nesse método, os dados são transformados de forma que apresentem

média igual a zero e desvio padrão igual a um, reduzindo diferenças de escala entre os atributos.

A segunda abordagem consistiu na utilização do método Min-Max Scaling, implementado pela classe `MinMaxScaler`, responsável por reescalar os atributos para um intervalo fixo entre 0 e 1. Essa técnica é frequentemente empregada quando se deseja preservar a distribuição relativa dos dados dentro de um intervalo padronizado.

Por fim, foi aplicada uma transformação logarítmica combinada à padronização Z-score, utilizando a função `log1p()` da biblioteca NumPy. Essa transformação teve como objetivo reduzir possíveis assimetrias e minimizar o impacto de valores extremos (*outliers*) presentes em atributos com distribuição mais dispersa. Após a transformação logarítmica, os dados também foram padronizados com `StandardScaler`.

É importante destacar que o processo de normalização foi realizado após a divisão entre treino e teste, evitando vazamento de informação (*data leakage*). Dessa forma, os parâmetros de escalonamento foram ajustados exclusivamente sobre os dados de treinamento (`fit_transform()`), sendo posteriormente aplicados ao conjunto de teste apenas por meio da transformação (`transform()`).

Esse cuidado metodológico é essencial em projetos de aprendizado de máquina, pois impede que informações do conjunto de teste influenciem indiretamente o treinamento do modelo, garantindo maior confiabilidade na avaliação do desempenho do classificador.

3.6 Implementação Inicial do Modelo KNN

Após a etapa de pré-processamento e normalização dos dados, foi implementado o primeiro modelo de classificação utilizando o algoritmo K-Nearest Neighbors (KNN), disponível na biblioteca Scikit-Learn.

Inicialmente, foi adotado um valor padrão de $k = 5$, representando a quantidade de vizinhos mais próximos considerados no processo de classificação de novas observações. Essa escolha inicial foi realizada com o objetivo de estabelecer uma linha de base (*baseline*) para comparação posterior com outras configurações do algoritmo.

O treinamento do modelo foi realizado utilizando o conjunto de treinamento previamente normalizado com a técnica Z-score (`StandardScaler`). Após essa etapa, o modelo foi avaliado sobre o conjunto de teste, permitindo verificar sua capacidade de generalização em dados não observados anteriormente.

Os resultados obtidos indicaram uma acurácia inicial de 91,66%, demonstrando que o algoritmo apresentou elevado desempenho já em sua configuração inicial. Esse

resultado sugere que os atributos geométricos do conjunto Dry Bean Dataset possuem boa capacidade discriminativa entre as diferentes variedades de feijão, favorecendo a separação das classes por meio de métricas de proximidade.

Apesar do desempenho satisfatório, a escolha do valor de k exerce influência direta na capacidade de generalização do modelo. Valores muito pequenos podem tornar o classificador excessivamente sensível a ruídos (*overfitting*), enquanto valores muito elevados podem simplificar excessivamente as fronteiras de decisão (*underfitting*). Dessa forma, tornou-se necessária uma investigação mais aprofundada da influência desse hiperparâmetro no desempenho do KNN.

3.7 Avaliação da Influência do Parâmetro K

Com o objetivo de investigar o impacto do hiperparâmetro k sobre o desempenho do classificador, foram realizados experimentos utilizando diferentes quantidades de vizinhos mais próximos. Os testes foram conduzidos considerando os seguintes valores de k : 1, 3, 5, 7, 9, 11, 13 e 15.

Para cada configuração, o modelo foi treinado utilizando o conjunto de treinamento normalizado e posteriormente avaliado sobre o conjunto de teste, registrando-se a acurácia obtida em cada cenário. Os resultados encontrados são apresentados na Tabela 1.

Valor de K	Acurácia (%)
1	89,86
3	91,52
5	91,66
7	91,37
9	91,63
11	91,85
13	91,88
15	91,88

Tabela 1 – Desempenho do algoritmo KNN para diferentes valores de k

Os resultados indicam que valores muito baixos de k , como $k = 1$, apresentaram desempenho inferior, possivelmente devido à elevada sensibilidade do modelo a ruídos e variações locais do conjunto de dados.

À medida que o número de vizinhos aumentou, observou-se uma melhoria gradual na capacidade de generalização do algoritmo, culminando nos melhores desempenhos para $k = 13$ e $k = 15$, ambos com acurácia de 91,88%.

Diante desse comportamento, optou-se inicialmente pela utilização de $k = 13$, uma vez que apresentou o mesmo desempenho máximo utilizando uma configuração menos complexa, favorecendo maior eficiência computacional.

Na próxima etapa, essa relação entre o valor de k e a acurácia do modelo foi analisada visualmente por meio de um gráfico de linha, permitindo identificar de maneira mais intuitiva a melhor parametrização do algoritmo.

3.8 Relação entre o Valor de K e a Acurácia do Modelo

Com o objetivo de visualizar o comportamento do algoritmo em diferentes configurações, foi construído um gráfico de linha relacionando os valores do hiperparâmetro k com a acurácia obtida pelo modelo.

O gráfico permitiu observar a evolução do desempenho do algoritmo à medida que a quantidade de vizinhos considerados no processo de classificação era alterada.

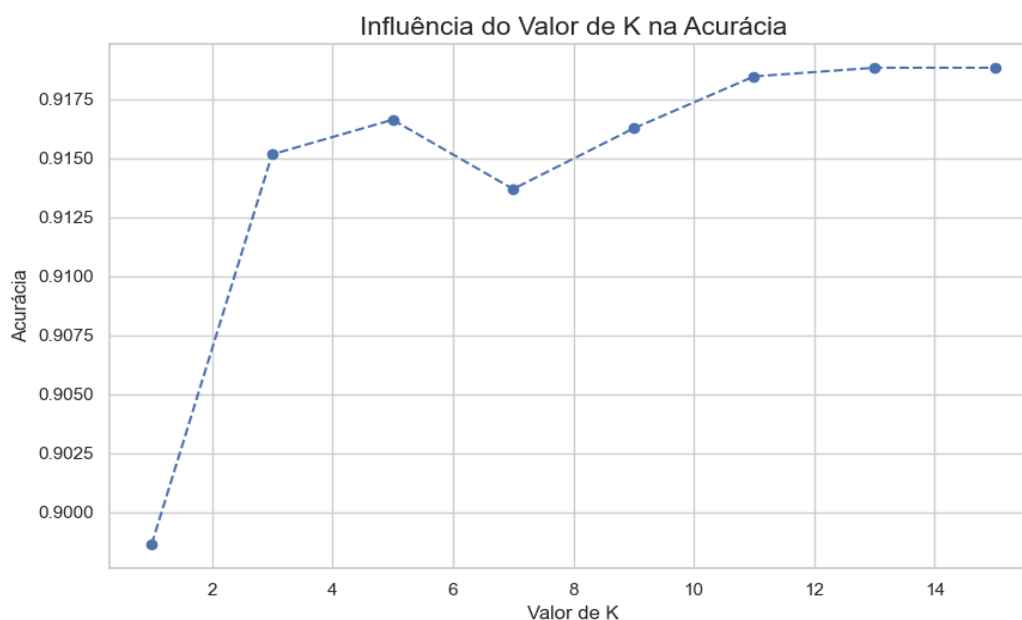


Figura 2 – Relação entre o valor de k e a acurácia obtida pelo modelo KNN.

A análise visual demonstrou que o desempenho do classificador apresentou crescimento significativo ao sair de $k = 1$, atingindo estabilidade para valores superiores a $k = 9$. Observou-se que os maiores desempenhos foram alcançados com $k = 13$ e $k = 15$, ambos apresentando 91,88% de acurácia. Esse comportamento sugere que o

algoritmo alcançou um ponto de equilíbrio entre sensibilidade local e capacidade de generalização.

Além disso, o gráfico evidencia que valores muito reduzidos de k podem tornar o modelo excessivamente sensível a pequenas variações do conjunto de treinamento, favorecendo fenômenos de sobreajuste (*overfitting*). Em contrapartida, valores demasiadamente altos tendem a suavizar excessivamente as fronteiras de decisão, podendo reduzir a capacidade discriminativa do classificador.

No caso do Dry Bean Dataset, verificou-se que valores intermediários e relativamente elevados de k proporcionaram melhor desempenho, indicando que a classificação dos grãos beneficia-se de uma análise baseada em múltiplos vizinhos próximos.

3.9 Identificação Visual da Melhor Parametrização do KNN

Após a construção do gráfico de desempenho, foi realizada a identificação visual da melhor configuração do algoritmo KNN. A partir da análise gráfica, verificou-se que o melhor desempenho foi obtido com $k = 13$, alcançando uma acurácia de 91,88% no conjunto de teste.

Embora o valor $k = 15$ também tenha apresentado desempenho equivalente, optou-se pela escolha de $k = 13$, considerando uma menor complexidade computacional e menor quantidade de vizinhos envolvidos no processo de decisão, sem perda de desempenho.

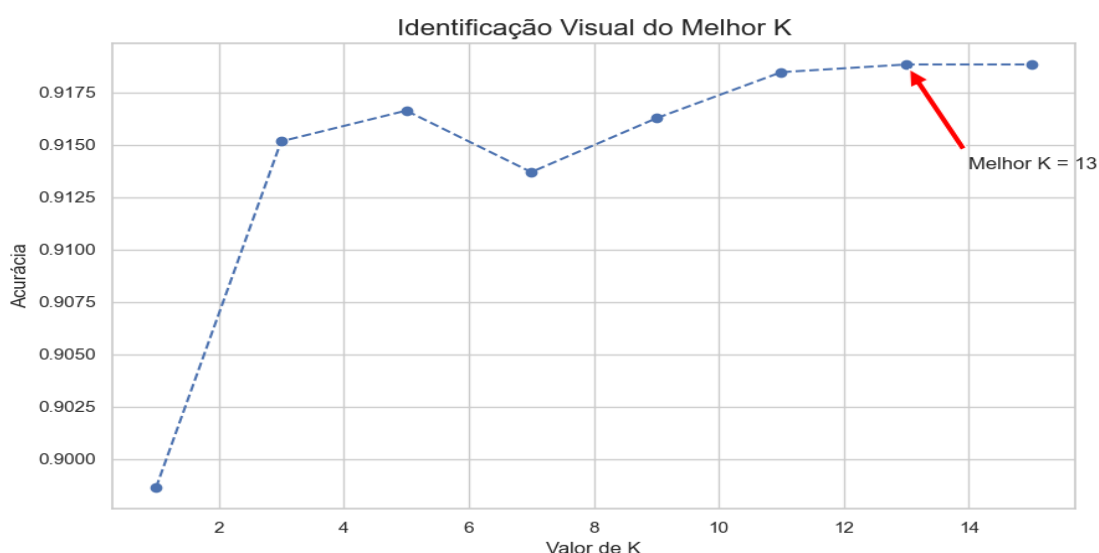


Figura 3 – Destaque visual da melhor parametrização do KNN.

Essa análise visual foi importante por fornecer uma interpretação intuitiva do comportamento do algoritmo, permitindo compreender como pequenas variações no

hiperparâmetro k podem impactar diretamente a capacidade de classificação do modelo.

Entretanto, embora a análise gráfica seja útil como ferramenta exploratória, ela não é suficiente para garantir que a configuração escolhida represente a melhor capacidade de generalização do modelo. Por esse motivo, tornou-se necessária a aplicação de técnicas mais robustas de avaliação estatística, como a validação cruzada estratificada.

3.10 Aplicação da Validação Cruzada Estratificada

Após a escolha inicial do valor de k , foi aplicada a técnica de validação cruzada estratificada (*Stratified K-Fold Cross Validation*), com o objetivo de obter uma avaliação mais robusta da capacidade de generalização do modelo.

A validação cruzada consiste em dividir o conjunto de dados em múltiplos subconjuntos (*folds*), alternando sucessivamente entre treinamento e validação. Dessa forma, cada amostra do dataset é utilizada tanto para treinamento quanto para validação ao longo do processo, reduzindo a dependência de uma única divisão treino-teste.

Neste projeto, foi utilizada uma estratégia de 5 *folds* estratificados, implementada por meio da classe *StratifiedKFold*, com embaralhamento dos dados (*shuffle=True*) e *random_state = 42* para garantir reprodutibilidade dos experimentos. A utilização da abordagem estratificada foi particularmente importante devido à natureza multiclasse do problema, pois garantiu a preservação proporcional das diferentes categorias de feijão em cada subconjunto de validação.

A Tabela 2 apresenta os resultados obtidos em cada *fold* da validação cruzada.

Fold	Acurácia
1	91,19%
2	92,91%
3	92,40%
4	92,95%
5	92,36%

Tabela 2 – Resultados da validação cruzada estratificada.

A média geral de desempenho obtida foi de 92,36%, com um desvio padrão de apenas 0,64%, indicando elevada estabilidade do modelo entre diferentes divisões dos dados.

Esse resultado é extremamente relevante, pois sugere que o classificador não apresentou forte dependência de uma única amostra de treinamento, demonstrando boa capacidade de generalização e reduzida variabilidade de desempenho.

Além disso, a proximidade entre a acurácia obtida no conjunto de teste (91,88%) e a média da validação cruzada (92,36%) reforça a consistência do modelo e reduz indícios de sobreajuste (*overfitting*).

3.11 Aplicação do Grid Search para Otimização de Hiperparâmetros

Embora a análise visual tenha indicado $k = 13$ como a melhor escolha inicial, foi utilizada uma estratégia automatizada de busca de hiperparâmetros com o objetivo de confirmar essa seleção e investigar possíveis combinações mais eficientes do algoritmo. Para isso, aplicou-se o método Grid Search, implementado pela classe GridSearchCV da biblioteca Scikit-Learn.

A busca foi realizada testando diferentes combinações dos seguintes hiperparâmetros:

- Número de vizinhos (*n_neighbors*): 1, 3, 5, 7, 9, 11, 13 e 15;
- Peso dos vizinhos (*weights*):
- uniform (peso igual para todos os vizinhos);
- distance (peso proporcional à distância);

Métrica de distância (*metric*):

- euclidean;
- manhattan.

O processo de busca utilizou validação cruzada estratificada de 5 *folds*, assegurando uma avaliação robusta de cada configuração testada. Os resultados do Grid Search indicaram a seguinte configuração ótima:

- Métrica: Euclidean
- Número de vizinhos: 13
- Pesos: Distance

Além disso, o melhor desempenho médio encontrado foi de 92,62% de acurácia.

Os resultados obtidos reforçam a consistência da análise visual realizada anteriormente, uma vez que o Grid Search confirmou a escolha de $k = 13$ como melhor valor do hiperparâmetro.

Entretanto, observou-se que a introdução de pesos baseados na distância (*distance weighting*) proporcionou um ganho adicional de desempenho, sugerindo que vizinhos mais próximos exercem maior relevância no processo de decisão do classificador.

Esse comportamento é coerente com a natureza do algoritmo KNN, pois amostras geometricamente mais próximas tendem a apresentar maior similaridade morfológica entre os grãos. Assim, a utilização do Grid Search permitiu não apenas validar a escolha visual do valor de k , mas também identificar automaticamente uma configuração mais robusta do modelo, maximizando seu desempenho preditivo.

3.12 Comparação do Desempenho entre Diferentes Técnicas de Normalização

Como o algoritmo K-Nearest Neighbors (KNN) é altamente sensível à escala dos atributos, realizou-se uma comparação entre diferentes estratégias de normalização com o objetivo de identificar qual abordagem proporcionaria melhor desempenho ao classificador.

Foram avaliadas três técnicas distintas:

- Z-score (StandardScaler);
- Min-Max Scaling (MinMaxScaler);
- Transformação logarítmica combinada ao Z-score (Log + StandardScaler).

A avaliação foi realizada utilizando o modelo otimizado obtido pelo Grid Search, empregando validação cruzada estratificada de 5 *folds* para garantir maior robustez estatística. Os resultados obtidos são apresentados na Tabela 3.

Técnica de Normalização	Acurácia Média (%)
Z-score	92,54
Min-Max	92,29
Log + Z-score	92,61

Tabela 3 – Comparação entre técnicas de normalização aplicadas ao KNN.

Os resultados demonstraram que todas as técnicas apresentaram desempenho elevado e relativamente próximo, evidenciando a robustez do algoritmo frente às diferentes estratégias de pré-processamento.

Entretanto, observou-se um desempenho ligeiramente superior da abordagem Log + Z-score, que alcançou 92,61% de acurácia média, tornando-se a melhor estratégia entre as alternativas avaliadas.

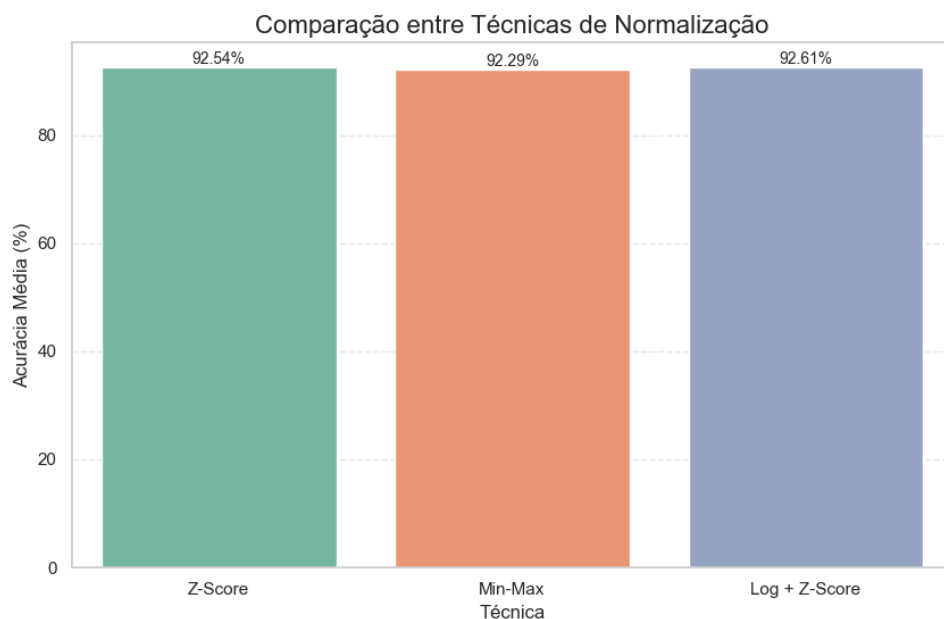


Figura 4 – Comparação do desempenho médio entre diferentes estratégias de normalização

Esse resultado sugere que a transformação logarítmica auxiliou na redução de possíveis assimetrias presentes em atributos geométricos mais dispersos, contribuindo para uma representação mais equilibrada do espaço de características antes da aplicação do algoritmo KNN.

Apesar disso, a diferença entre as abordagens foi relativamente pequena, indicando que o conjunto Dry Bean Dataset já apresenta boa qualidade estrutural e baixa sensibilidade extrema à escala dos atributos após normalização. Além disso, verificou-se que o Min-Max Scaling apresentou desempenho ligeiramente inferior (92,29%), embora ainda tenha mantido excelente capacidade preditiva.

De maneira geral, os resultados reforçam a importância da etapa de normalização em algoritmos baseados em distância, uma vez que o desempenho do KNN depende diretamente da forma como os atributos são escalonados.

3.13 Matriz de Confusão e Métricas de Classificação

Após a definição da melhor configuração do modelo, realizou-se uma avaliação detalhada do desempenho do classificador utilizando a matriz de confusão e o classification report, permitindo analisar o comportamento do modelo individualmente para cada classe do problema.

A matriz de confusão possibilita comparar as classes reais com as classes previstas pelo algoritmo, evidenciando acertos e erros de classificação.

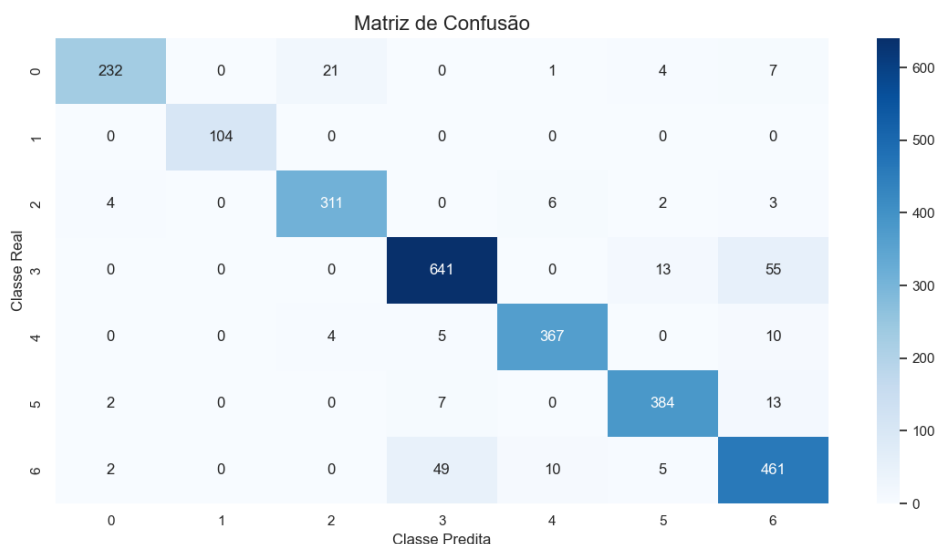


Figura 5 – Matriz de confusão do modelo KNN otimizado.

A análise visual da matriz indicou elevado desempenho geral do modelo, com forte concentração de previsões na diagonal principal, representando classificações corretas.

Observou-se que algumas categorias apresentaram desempenho extremamente elevado, como a classe BOMBAY, que obteve classificação praticamente perfeita, com 100% de precisão, recall e F1-score.

Em contrapartida, verificou-se maior dificuldade de classificação na classe SIRA, que apresentou desempenho relativamente inferior em comparação às demais categorias, como podemos ver na Tabela 4 o resumo das métricas por classe.

Classe	Precision	Recall	F1-score
BARBUNYA	0,97	0,88	0,92
BOMBAY	1,00	1,00	1,00
CALI	0,93	0,95	0,94
DERMASON	0,91	0,90	0,91
HOROZ	0,96	0,95	0,95
SEKER	0,94	0,95	0,94
SIRA	0,84	0,87	0,86

Tabela 4 – Métricas de classificação por categoria de feijão.

Esse comportamento sugere uma maior semelhança morfológica entre a classe SIRA e outras variedades de feijão presentes no dataset, aumentando a probabilidade de confusão durante o processo de classificação.

De forma agregada, o modelo apresentou:

- Acurácia geral: 92%;
- Macro Average: 93%;
- Weighted Average: 92%.

Esses resultados indicam elevado equilíbrio no desempenho entre as diferentes classes, demonstrando que o classificador apresentou boa capacidade de discriminação mesmo em um problema multiclasse composto por sete categorias distintas.

A proximidade entre os valores de precision, recall e F1-score também sugere um comportamento relativamente balanceado do modelo, reduzindo indícios de viés excessivo em favor de determinadas categorias.

4. Conclusões

Neste projeto foi desenvolvido um modelo de classificação supervisionada utilizando o algoritmo K-Nearest Neighbors (KNN) aplicado ao Dry Bean Dataset, com o objetivo de classificar diferentes variedades de grãos de feijão a partir de atributos geométricos extraídos por processamento de imagens.

Os resultados obtidos demonstraram que o modelo apresentou elevado desempenho na tarefa de classificação, alcançando 91,66% de acurácia inicial, com melhora após o ajuste de hiperparâmetros e aplicação de validação cruzada, atingindo aproximadamente 92,62% de acurácia no melhor cenário encontrado.

A análise do parâmetro k mostrou que valores intermediários proporcionaram melhor capacidade de generalização, sendo $k = 13$ identificado como a melhor escolha tanto pela análise visual quanto pelo Grid Search. Além disso, verificou-se que a normalização dos dados teve impacto relevante no desempenho do modelo, com a abordagem Log + Z-score apresentando o melhor resultado médio.

De forma geral, conclui-se que o algoritmo KNN apresentou excelente capacidade de classificação para o problema proposto, demonstrando ser uma abordagem eficiente e confiável para tarefas de reconhecimento automático de padrões agrícolas.

5. Próximos passos



Como continuidade deste projeto, recomenda-se inicialmente testar outros algoritmos de classificação, como Random Forest e Redes Neurais Artificiais, a fim de comparar seus desempenhos com o KNN e verificar a possibilidade de obter ganhos adicionais de acurácia. Além disso, pode-se aprofundar a etapa de otimização de hiperparâmetros, explorando intervalos maiores para o valor de k , novas métricas de distância e diferentes estratégias de ponderação dos vizinhos.

Por fim, sugere-se a aplicação do modelo em cenários mais próximos do ambiente real, utilizando novas amostras obtidas diretamente por processamento de imagens agrícolas, permitindo avaliar a capacidade de generalização do sistema fora do conjunto de treinamento original.